



Spring Security: Authentication and Authorization for Modern Applications

Section 1: Learn

What is Spring Security?

Spring Security is a comprehensive framework that provides authentication and authorization capabilities for Java applications. It helps safeguard your web apps and APIs from common vulnerabilities by using filters, providers, and secure contexts.

Why Use Spring Security?

- **Authentication & Authorization:** Verifies identity and grants access.
- **Flexible & Extensible:** Supports form login, basic auth, OAuth2, and JWT.
- **Secure REST APIs:** Ideal for stateless architectures and microservices.
- **Defense in Depth:** Protects against CSRF, session fixation, and more.

Key Concepts

- **Authentication:** Who are you? (username/password, OAuth2 token, etc.)
- **Authorization:** What are you allowed to do? (role, permission, ACL)
- **SecurityContextHolder:** Stores the authenticated user details.
- **Filter Chain:** Intercepts every request and applies security rules.

Interesting Anecdote

JWT (JSON Web Token) became popular in stateless REST architectures because they reduce server load and simplify authentication across microservices—paving the way for scalable APIs used in mobile and cloud platforms.



Section 2: Practice

Authentication vs Authorization

- **Authentication:** Confirms identity.
 - Username/Password
 - OAuth tokens
 - Multi-Factor Authentication
- **Authorization:** Grants permission.
 - Role-based access
 - URL restrictions
 - Method-level security

JWT (JSON Web Token) Overview

- **Header:** Algorithm + type
- **Payload:** Claims like user info, roles, expiration
- **Signature:** Ensures token integrity

Example Token:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Steps to Implement JWT Authentication in Spring Boot

1. Setup Project

Dependencies: Spring Web, Spring Security, Spring Data JPA, jjwt

2. Create User Entity & Repository

```
@Entity
```

```
public class User {
```



```
private String username;
private String password;
}
@Repository
public interface UserRepository extends JpaRepository<User, Long> {}
```

3. Configure Security

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.csrf().disable()
            .authorizeRequests()
            .antMatchers("/auth/**").permitAll()
            .anyRequest().authenticated();
        return http.build();
    }
}
```

4. Implement JWT Utility

```
public class JwtUtil {
    public String generateToken(UserDetails userDetails) { ... }
    public boolean validateToken(String token, UserDetails userDetails) { ... }
}
```



5. Create Filter to Validate JWT

```
public class JwtAuthFilter extends OncePerRequestFilter {  
    protected void doFilterInternal(...) {  
        // Extract token, validate, and set context  
    }  
}
```

6. Add Authentication and Registration Endpoints

```
@PostMapping("/auth/login")  
public ResponseEntity<?> login(@RequestBody LoginRequest login) {  
    // Authenticate user and return token  
}
```

Advantages of JWT

- Stateless & scalable
- Works across domains
- Compact and signed
- Reduces DB queries

JWT Best Practices

- Use strong secret keys
- Short token lifetimes
- Rotate keys periodically
- Avoid storing tokens in localStorage
- Use HttpOnly cookies when possible



Section 3: Know More

FAQs

Q1: What is the difference between Basic Auth and JWT?

- **Answer:** Basic Auth sends username/password on each request. JWT uses a token that's generated once and sent with each request.

Q2: Can Spring Security support multiple authentication types?

- **Answer:** Yes, it can handle Basic, Form Login, OAuth2, and JWT simultaneously with proper configuration.

Q3: Where should I store JWT tokens in the frontend?

- **Answer:** Use HttpOnly cookies for web apps. Avoid localStorage due to XSS risks.

Q4: How do I expire or revoke a JWT token?

- **Answer:** Add short expiration times and use refresh tokens. Store token blacklists on the server if revocation is needed.

Q5: What are common pitfalls to avoid with JWT?

- **Answer:** Not validating signatures, long token lifespan, weak keys, storing tokens insecurely, skipping HTTPS.

Next Steps:

- Implement JWT in your own Spring Boot app.
- Experiment with role-based access.
- Learn about token refresh and logout strategies.